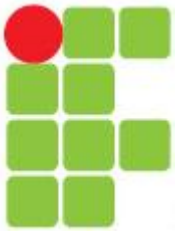
INSTITUTO FEDERAL DE ALAGOAS – IFAL / CAMPUS MACEIÓ
Curso Bacharelado em Sistemas de Informação
Disciplina de Qualidade de Software

Prof. MSc. Ricardo Nunes
Ricardo (arroba) ifal.edu.br



Atividade Avaliativa — Testes no R Book Lib

Objetivo geral

Escrever **testes de unidade** com **Jest** para as regras de negócio do Sistema de Biblioteca R Book Lib, evoluir o sistema com novas funcionalidades testadas e relatar o resultado.

O R Book Lib é um sistema pequeno, mas organizado como um projeto real: modelos, repositório, serviço e interface de console. Os testes devem exercitar as regras de negócio por meio do `LibraryService`. **A interface de console não é testada.**

Organização

- A atividade é feita **em grupo de até 4 pessoas**.
- Vale **10 pontos**, distribuídos entre as quatro partes (veja Avaliação).

Arquivos associados

1. `1-descricao-rbl.md`: descrição do sistema, regras de negócio e mensagens de erro
2. `2-historias-usuarios-rbl.md`: histórias de usuário implementadas no sistema
3. `3-evolucoes-rbl.md`: novas funcionalidades propostas para a evolução do sistema
4. `4-historias-usuarios-evolucoes-rbl.md`: histórias de usuário das evoluções
5. `r-book-lib.zip`: código-fonte do sistema, pronto para rodar, com o modelo do relatório

Preparação

Descompacte o `r-book-lib.zip` e, na pasta do projeto, execute:

```
npm install
npm test
```

O projeto traz um teste de exemplo, que deve passar. Se passar, o ambiente está funcionando.

Comando	O que faz
`npm test`	Executa os testes
`npm test -- -t "parte do nome"`	Executa só os testes cujo nome contém o texto
`npm test -- --randomize`	Executa os testes em ordem aleatória
`npm run coverage`	Executa os testes e mostra o relatório de cobertura

Comando	O que faz
`npm start`	Abre o sistema no console

O projeto usa **ES Modules** (`import`/`export`), e o `package.json` já está configurado para o Jest executá-los.

Estrutura do projeto

```
r-book-lib/
├─ package.json
├─ README.md
├─ relatorio.md                    # modelo do relatório (Parte 4)
├─ src/
│  ├─ app.js                       # ponto de entrada (console)
│  ├─ ui/
│  │  └─ console-ui.js
│  ├─ models/
│  │  ├─ book.js
│  │  └─ user.js
│  ├─ repo/
│  │  └─ in-memory-repository.js
│  └─ services/
│     └─ library-service.js
└─ tests/
   └─ library-service.test.js      # teste de exemplo
```

Parte 1 — Testes do sistema atual (4,0 pontos)

Escreva os testes de unidade do `LibraryService`, cobrindo as regras de negócio **RN01 a RN08** descritas em `1-descricao-rbl.md`.

Os casos de teste não são fornecidos: **derivar os casos a partir das regras faz parte da atividade.** Para cada regra, pergunte:

- qual é o comportamento quando a regra é respeitada?
- qual é o comportamento quando ela é violada, e qual é a mensagem de erro?
- quais são os **limites** da regra (o último valor aceito e o primeiro recusado)?

Exigido:

- **No mínimo 15 testes.**
- Todas as regras RN01 a RN08 com pelo menos um teste do caso válido e um do caso inválido.
- Os limites das regras que têm valores numéricos (quantidade de exemplares e quantidade de empréstimos).
- Nos casos de erro, verificar a **mensagem** com `toThrow`.
- Nos casos de sucesso, verificar o **estado** depois da operação (quantidade disponível, lista de empréstimos, listagem).

Qualidade dos testes. Além de existirem, os testes são avaliados por estes critérios:

Critério	O que se espera
Nomes	O nome do teste descreve o comportamento verificado
Independência	Cada teste prepara o seu próprio estado e passa sozinho e em qualquer ordem
Verificação	Cada teste compara o resultado com o valor esperado, escrito diretamente
Simplicidade	Sem `if`, `for` ou `try/catch` dentro dos testes
Preparação	Sem blocos de preparação repetidos (use `beforeEach` ou funções auxiliares)
Organização	Testes agrupados com `describe`

O código pode conter defeitos

O código distribuído **pode conter defeitos** em relação às regras documentadas. A referência do comportamento correto é o documento `1-descricao-rbl.md`, e não o código.

Se um teste escrito a partir de uma regra falhar:

1. confira se o teste está correto em relação à regra;
2. mantenha o teste e **registre o defeito no relatório** (regra, teste, comportamento esperado e comportamento obtido);
3. **corrija o código**, de modo que a suíte entregue fique toda verde.

Não altere um teste só para ele passar: um teste que acompanha o defeito do código não verifica nada.

Parte 2 — Evoluções (3,0 pontos)

Implemente **duas** das novas funcionalidades descritas em `3-evolucoes-rbl.md` e `4-historias-usuarios-evolucoes-rbl.md`, **com testes de unidade**:

1. **Uma evolução com data, obrigatória:** NF02 (registrar data de empréstimo e devolução) **ou** NF03 (bloquear empréstimos atrasados).
2. **Uma evolução de livre escolha:** NF01, NF05, NF07 ou NF08.

Exigido:

- Na evolução com data, a data atual deve ser **recebida como parâmetro**, com a data do momento como valor padrão. Os testes passam **datas fixas** e não podem depender do dia ou da hora em que são executados.
- Os critérios de aceitação de cada história escolhida devem ter testes.
- Os testes da Parte 1 devem continuar passando.
- Os mesmos critérios de qualidade da Parte 1 valem aqui.

As funcionalidades NF06 e NF09 gravam arquivos em disco e **não contam** para esta parte.

A ordem de trabalho é livre: os testes podem ser escritos antes ou depois da implementação.

Parte 3 — Cobertura (1,0 ponto)

Meça a cobertura dos testes com:

```
npm run coverage
```

O projeto já está configurado para medir os arquivos de `src/models/`, `src/repo/` e `src/services/`. Os arquivos `src/app.js` e `src/ui/` ficam fora da medição.

Exigido:

- **Pelo menos 90% de linhas e 90% de ramos** (colunas `% Lines` e `% Branch`). Abaixo disso, o próprio comando acusa o erro.
- No relatório, a tabela de cobertura e a **justificativa de cada trecho não coberto**.

A cobertura serve para encontrar o que ficou sem teste. Um trecho pode ficar descoberto por um bom motivo, desde que o motivo esteja explicado.

Parte 4 — Relatório (2,0 pontos)

Preencha o arquivo `relatorio.md`, que acompanha o código. Ele tem quatro perguntas:

1. **Defeitos encontrados**
2. **Testabilidade**
3. **Cobertura**
4. **O que ficou sem teste, e por quê**

Cada resposta deve apontar para o código do grupo (nome do teste, arquivo, trecho). Respostas genéricas, que serviriam para qualquer projeto, não pontuam.

Entrega

Um arquivo compactado com o projeto, **sem a pasta `node_modules`**, contendo:

- o código-fonte, com as correções e as evoluções;
- os testes;
- o `relatorio.md` preenchido, com o nome dos integrantes.

O projeto deve funcionar com `npm install` seguido de `npm test`.

Avaliação

Parte	Pontos	O que é avaliado
1. Testes do sistema atual	4,0	Casos derivados das regras, limites, qualidade dos testes e tratamento dos defeitos encontrados
2. Evoluções	3,0	Funcionamento das duas evoluções, testes e uso de data recebida por parâmetro
3. Cobertura	1,0	Meta atingida e trechos não cobertos justificados

Parte	Pontos	O que é avaliado
4. Relatório	2,0	Respostas às quatro perguntas, com evidências do código do grupo
Total	**10,0**	